ROS2 Assignment Report

Pranav Bagrecha
Software Subsystem, MRT, IITB

October, 2026

1 Issues

The issues encountered during the completion of the assignment, along with the implemented solution, are documented below.

1.1 Task 1: Workspace and Packages

No significant errors or issues were encountered during the implementation of this assignment as the task was only to create packages and the code was provided in the assignment itself.

1.2 Task 2: Power Publisher

1. **Issue: Integer Stored in Float**
   Cause: While initial declaration, temperature and battery were declared as 100, 20 respectively, and then it was parsed in `Float32MultiArray` which conflicts the data type.
   Solution: Declare the temperature and battery as 100.0 and 20.0, respectively, and be cautious about data types.

1.3 Task 3: Health and Monitor Node

1. **Issue: Publishing in Health Node**
   Cause: Got confused between `string` and `String` so I directly published it as `self.pub.publish("MSG")`
   Solution: Publish it as `String("MSG")`

1.4 Task 4: Launch File

1. **Issue: Unable to filter logs to display only those from `monitor_node`**
   Cause: I expected that writing `output='screen'` would mean that only the logs of that node would be displayed but it is not like that.
   Solution: There is another command that uses `-ros-args` to do that.

1.5 Task 5: Custom Message and Odometry

1. **Issue: Difficulties in building `mars_msgs` package**
   Cause: Did not update `package.xml` and `CMakeLists.txt` properly
   Solution: Reread the documentation and corrceted syntax and including errors accordingly

2. **Issue: Errors in calling parameters**
   Cause: Used `self.get_logger().warn(...)`
   Solution: According to some new update, `warn()` is now `warning()` so replaced it with `self.get_logger().warning(...)`

1.6 Task 6: Parameters

1. **Issue: While reading the values of parameters**
   Cause: I assumed that parameters parameters used in YAML file may not be required to declared again in node and its value can be read directly but that is not how things work.
   Solution: Declare parameters in node before using it.

1.7 Task 7: Close the Loop

1. **Issue: Even without implementing any specific QoS, `monitor_node` was able to read previous value published on `rover/health`**
   Cause: I always declared publisher as `self.pub = self.create_publisher(<type>, '<name>', 10)`. So the 10 written did the job.

2. **Issue: Math runtime error**
   Cause: While implementing 'Better Odometry', I did not bother about the edge case where angular velocity is zero, so it gave divide by 0 error.
   Solution: Add an edge case for situation when angular velocity is zero.

1.8 Task 8: Mission Control

1. **Issue: Multiple Conflict between `async` and `spin` causing a bunch of runtime errors**
   Cause: Did many logical errors and caused conflict between different functions. **Solution:** Rewrote the code over the template code available in documentation.

2. **Issue: Unoptimised Path towards Shelter**
   Cause: The program for reaching the destination only made rover go along the axis and hence consumed a huge amount of battery and at times didn't reach the destination when it could have reached if straight line path was followed.
   Solution: Implemented the straight line path navigation.

3. **Issue: Heading Correction while moving towards Shelter**
   Cause: The heading was set only once during initialization, but at times it had errors and overturned due to finite amount of time between successive checks. This caused the rover to significantly deflect from intended target if the distance was large enough.
   Solution: Implemented heading checks and correction after each loop.

4. **Issue: Overrunning over Shelter**
   Cause: The speed of rover was set at speed limit of the rover as in `odom_node`, but again due to finite interval between successive checks, the rover sometimes overran over the shelter, then turned 180° and then again overran and was stuck in this infinite loop.
   Solution: Reduce the speed of rover when the shelter was close enough.

1.9 Task 9: Two Rovers and a Bag File

1. **Issue: Conflict between topic names when two rovers are ran simultaneously**
   Cause: Absolute naming of topics, services and actions while declaring them by using \ while declaring.
   Solution: Remove the preceding \ from the names, which makes their names relative and they get distinguished by namespace.

2. **Issue: Recording**
   Cause: Due to some reason, wildcards like `ros2 bag record -o rover_bag -topics /rover/* /cmd_vel` gave not recognised error.
   Solution: Listed all the topics as `ros2 bag record -o rover_bag -topics /cmd_vel /rover/health /rover/monitor /rover/odom /rover/power` .

2 How a ROS2 Service Works

ROS2 services provide a communication mechanism based on a request-response model. A client sends a request to a server, and the server processes the request and sends back a response.

2.1 Service Server

A service server declares/creates a particular service and waits for requests from clients. When a request is received, the server processes it and returns an appropriate response.

2.2 Service Client

A service client is responsible for sending requests to a service server. It waits for the server to process the request and return a response.

2.3 Service Call Process

The working of a ROS 2 service can be explained using the `/rover/set_mode` service:

1. The service server (here `drive_node`) starts and advertises the `/rover/set_mode` service.
2. A client (here `navigator_node`) sends a request to change the rover's operating mode.
3. The server receives the requested mode and processes the request.
4. The server updates the rover's mode if the request is valid.
5. The server sends a response to the client, indicating whether the operation was successful.
6. The client receives and displays the response and proceeds with the further execution if positive response is received based on the code.

2.4 Difference Between Services, Topics, and Actions

ROS 2 provides three main communication mechanisms:

- **Services:** Services use request-response communication. They are suitable for short operations that require a response, such as changing the rover's operating mode.

- **Topics:** Topics use a publish-subscribe communication model. Publishers publish messages, and subscribers receive them without requiring a direct response. They are suitable for sensor data and continuous updates since they can be stored, accessed and published by multiple other nodes as required.
- **Actions:** Actions are designed for long-running tasks. They support sending goals, providing continuous feedback, and returning a final result. They also allow goals to be cancelled while executing based on other parameters, if any.

3 Comments on the Assignment

Overall, the assignments provided practical experience with ROS2 communication mechanisms and helped improve my understanding of robotics software development.

3.1 Parts That Were Easy

Earlier parts up to Task 4 where quite easy and straight-forward since almost the whole code was already written in the assignment with only few logical steps that had to be written which was already practiced to a great extent in ARIA Assignment earlier.

3.2 Parts That Were Difficult

Task 7 and 8 were quite difficult as they involved handling multiple nodes, topics, services and actions. Considerable amount of time needed to be invested in these task to remove conflicts between various topics and topic publishers

3.3 Unclear Aspects

It would have been great if the part of Task 5 introducing Custom Messages contained more syntax and changes that had to be done in `package.xml` and `CMakeLists.txt` since I personally had to spend considerable time figuring the syntax out of documentations.